

Can we teach a model how to find what it needs?

A recursive language model (RLM) can use tools and ask helper models to solve parts of a task.

For this study, we returned to a basic skill.

Can a small model look up missing information and use it to finish a task?

Example: Make a wooden pickaxe in a crafting game.

1. The model asks which ingredients it needs.
2. It makes the parts and then the pickaxe.
3. The game checks whether the pickaxe actually exists.

Simplified illustration of TextCraft, our test environment. These experiments use one model with tools, not recursive helper calls.

We compared two kinds of worked examples.

We trained copies of the same model to imitate the next action in an example. This is **supervised fine-tuning**.

The teacher already knows the recipes.

It chooses the route using that knowledge, then shows the steps.

“Look up planks. Make planks.
Continue along the planned route.”

The teacher discovers the recipes.

It chooses the route by looking up which parts are needed.

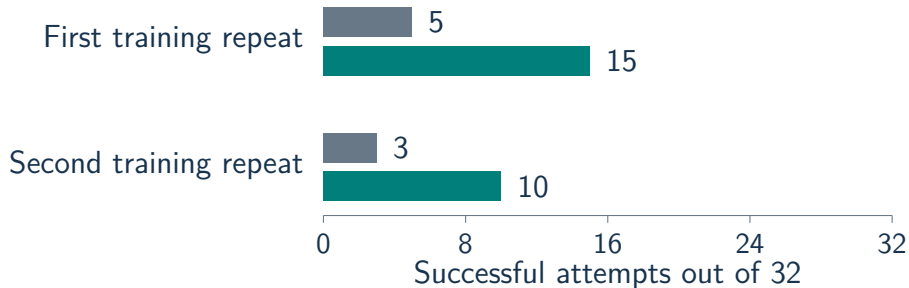
“Look up the pickaxe. Discover its parts. Look up their recipes.”

Both show recipe lookups. They differ in how the route is chosen.

Simplified examples, not literal training records. Both use the same 32 training tasks and 366 demonstrated steps. Their histories and amount of text differ.

Examples that discover the route worked better.

We trained each kind of model twice and tested them on the same tasks.



■ Teacher knows recipes ■ Teacher discovers recipes

Sixteen evaluation tasks, two attempts each. The original examples were corrected for a known quantity error. This correction did not remove the advantage.

The advantage also appeared when recipes changed.

We kept the goals but changed the recipes and starting supplies. Each cell below shows successful attempts out of 16.

Knows recipes $\longrightarrow$ Discovers recipes		
Changed task setting	First training repeat	Second training repeat
A	4 $\longrightarrow$ 7	3 $\longrightarrow$ 6
B	1 $\longrightarrow$ 8	1 $\longrightarrow$ 8
C	3 $\longrightarrow$ 8	1 $\longrightarrow$ 10

The direction repeats, but this is still one kind of game.

Important limit: these comparisons use the original examples before the quantity correction. Testing corrected examples with changed recipes is the next control.

Next: separate choosing a step from carrying it out.

A small change to the model's tools

The model chooses what to make. Code translates a recipe it has already looked up into the exact tool command.

Example: The model says “make two sticks.” Code fills in the ingredient amounts from the known recipe.

We can compare this with making the model fill in every command detail itself. Does it finish more tasks, including tasks with changed recipes?

Discussion: Is this a useful step toward teaching models to plan and divide larger tasks?

Proposed, not yet a result. Simply changing the reward-based training rule did not improve success: both tested versions solved 15/32 attempts.